PRACTICAL ON Maven

Create a Maven Project

Option 1: Using Maven Archetype

Run the following command:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=my-app1 -
DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

- `groupId`: Identifies the group (e.g., organization).
- `artifactId`: The name of the project.
- `archetypeArtifactId`: Template for the project structure.

To unserdand the above command :

1. **`mvn archetype:generate`**

- **`mvn`**: This is the Maven command-line tool.
- **`archetype:generate`**: This tells Maven to generate a project using a Maven archetype. An archetype is a template for creating projects with a predefined structure

2. **`-DgroupId=com.example`**

- **`-DgroupId`**: This specifies the group ID for the project.
- **`com.example`**: The value of the `groupId`. This typically represents the organization or company creating the project (e.g., a domain name in reverse order).

3. **`-DartifactId=my-app1`**

- **`-DartifactId`**: This specifies the artifact ID for the project.
- **`my-app1`**: The value of the `artifactId`. This is usually the name of your project and will be the base name of the generated `.jar` or `.war` file.

4. **`-DarchetypeArtifactId=maven-archetype-quickstart`**

- **`-DarchetypeArtifactId`**: Specifies which Maven archetype to use for project creation.
- **`maven-archetype-quickstart`**: The value of the `archetypeArtifactId`. This is a commonly used archetype that creates a simple Java project with a basic directory structure and a sample class.

5. **`-DinteractiveMode=false`**

- **`-DinteractiveMode`**: Determines whether Maven should prompt you for input during the process.

- **`false`**: Disables interactive mode, meaning Maven uses the specified options (`groupId`, `artifactId`, etc.) without prompting.

This command is used to quickly bootstrap a new Java project with Maven, giving you a starting point for development.

2. Navigate to the project directory:

   ```
   cd my-app
   ```

Option 2: Manually Create a Basic Maven Project

Create the following directory structure manually:

```
css
Copy code
my-app1/
├── pom.xml
└── src/
    ├── main/
    │   └── java/
    │       └── com/
    │           └── example/
    │               └── App.java
    └── test/
        └── java/
            └── com/
                └── example/
                    └── AppTest.java
```

3. Edit the `pom.xml`

The `pom.xml` is the heart of your Maven project. Here's a minimal example:

```
xml
Copy code
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>my-app1</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

4. Write a Simple Java Program

Main class (`src/main/java/com/example/App.java`):

```
java
Copy code
package com.example;

public class App {
    public static void main(String[] args) {
        System.out.println("Hello, Maven!");
    }
}
```

Test class (`src/test/java/com/example/AppTest.java`):

```
java
Copy code
package com.example;

import org.junit.Test;
import static org.junit.Assert.assertTrue;

public class AppTest {
    @Test
    public void shouldAnswerWithTrue() {
        assertTrue(true);
    }
}
```

5. Build and Run the Project

1. **Compile the code**:

   ```
   mvn compile
   ```

2. **Run tests**:

   ```
   mvn test
   ```

3. **Package the application**:

   ```
   mvn package
   ```

   This creates a `my-app1-1.0-SNAPSHOT.jar` in the `target/` directory.

4. **Run the application**:

   ```
   java -cp target/my-app1-1.0-SNAPSHOT.jar com.example.App
   ```

   Output:

   ```
   Copy code
   Hello, Maven!
   ```

To add a dependency (e.g., a logging library like Log4j), include it in the `<dependencies>` section of your `pom.xml`:

```
xml
Copy code
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>2.20.0</version>
</dependency>
```

Run:

```
bash
Copy code
mvn clean install
```

This is a simple yet comprehensive example to get started with Maven. Let me know if you'd like to explore specific Maven features further!